Programowanie współbieżne w C++

Lista zadań nr 9

Na pracownię 30 kwietnia 2026 (i być może późniejsze)

Zadanie 1 (7pktów). Współbieżna kolejka (prawie) bez blokad.

Zaimplementuj kolejkę opartą na liście jednokierunkowej ze strażnikiem i zarządzaniem pamięcią przeznaczoną na węzły za pomocą smart pointerów. Dostarczona przez bibliotekę standardową implementacja tychże, to jedyne miejsce, w którym dopuszczamy blokady. Wymagania:

1. Strażnik (dummy node): Kolejka musi być inicjalizowana z pustym węzłem strażnikiem, na który początkowo wskazują zarówno wskaźnik `head`, jak i `tail`. Upraszcza to znacząco synchronizację operacji na pustej kolejce.

2. Operacja `enqueue(T data)`: Dodawanie elementu polega na dołączeniu nowego węzła na koniec listy, a następnie przesunięciu wskaźnika `tail`. Zastanów się, co się stanie, jeśli wątek zostanie wywłaszczony pomiędzy tymi dwoma krokami. Zaimplementuj mechanizm "wzajemnej pomocy" – wątek, który zauważy, że wskaźnik `tail` jest w tyle za faktycznym końcem listy, powinien pomóc go przesunąć za pomocą operacji `compare_exchange_weak/strong`, zanim spróbuje dodać swój własny element.

3. Operacja `dequeue()`: Pobieranie elementu (zwracające np. `std::optional<T>` lub `std::shared_ptr<T>`) przesuwa wskaźnik `head` i odczytuje wartość. Pamiętaj o weryfikacji, czy kolejka nie jest pusta.

W operacjach atomowych zastosuj domyślny porządek `std::memory_order_seq_cst`. Przetestuj rzetelnie swoją implementację.

Wskazówka: The Art of Multiprocessor Programming 2e, r. 10.5.

Zadanie 2 (3pkty). Optymalizacja porządków pamięciowych w kolejce z poprzedniego zadania.

Zoptymalizuj porządki dostępów do pamięci w swoim rozwiązaniu zadania 1. Uzasadnij, że w Twoim rozwiązaniu nie ma wyścigów.

Zadanie 3 (7pktów). Współbieżny stos z tablicą eliminacji.

W zadaniu 4 z Listy 5 zaimplementowaliście współbieżny stos (prawie) bez blokad. W warunkach silnej rywalizacji, wierzchołek stosu staje się wąskim gardłem. Operacje compare_exchange często kończą się niepowodzeniem, zmuszając wątki do powtarzania pętli i marnowania cykli procesora. Rozwiązaniem tego problemu jest struktura Elimination Backoff Stack, która wykorzystuje tablicę eliminacji, złożoną z obiektów LockFreeExchanger (zaimplementowany przez Was w zadaniu 5 na Liście 5).

Opis algorytmu: Kiedy wątek próbuje wykonać operację push(T) lub pop() i doświadczy rywalizacji (zobaczy, że jego operacja CAS na zmiennej top się nie powiodła), zamiast natychmiast próbować ponownie, wycofuje się (backoff) do tablicy eliminacji. Tzn. losuje jeden ze slotów w tablicy eliminacji (w każdym slocie znajduje się instancja LockFreeExchanger). Jeśli wykonuje push, oferuje swój element "na wymianę" przez krótki czas (timeout). Jeśli wykonuje pop, oczekuje na element od partnera również przez ustalony krótki czas. Jeśli w tym samym slocie i w tym samym czasie spotkają się wątek push i wątek pop, wymieniają się wartością (element zostaje przekazany bezpośrednio z wątku do wątku). Obie operacje kończą się sukcesem bez jakiejkolwiek modyfikacji wierzchołka głównego stosu. Jeśli wymiana w tablicy się nie powiedzie (wystąpił timeout), wątek wraca do głównej pętli i ponownie próbuje zmodyfikować top stosu.

Zaimplementuj klasę EliminationBackoffStack<T>. Zintegruj z nią swoją klasę LockFreeExchanger z Listy 5. Dobierz optymalny rozmiar tablicy eliminacji (poeksperymentuj – zbyt mała nie rozładuje ruchu, zbyt duża sprawi, że wątki rzadko będą na siebie trafiać). Napisz testy wydajnościowe. Porównaj czas wykonania 1 miliona operacji (50% push, 50% pop) na tej strukturze ze zwykłym stosem z Listy 5. Przetestuj to dla 1, 2, 4, 8 i maksymalnej dostępnej na Twojej maszynie liczby wątków sprzętowych. Krótko skomentuj wyniki testu w pliku źródłowym. W jakich warunkach zauważasz największy zysk z zastosowania tablicy eliminacji?

Wskazówka: The Art of Multiprocessor Programming 2e, r. 11.4.